Mini-relatório – Estrutura de Dados (Entrega 2)

Introdução

Na Entrega 1, a aplicação C# lia os alunos do banco (AlunoDB) e exibia uma List paginada, sem busca, filtro ou ordenação. Nesta entrega, os dados continuam sendo lidos do banco uma única vez, mas agora ficam organizados em memória na estrutura mais adequada a cada operação do menu.

Estruturas aplicadas

List com paginação por índice

```
int inicio = (pagina - 1) * tamanho;

return lista.GetRange(inicio, Math.Min(tamanho, lista.Count - inicio));
```

O acesso por índice custa O(1). A página é calculada direto pela fórmula, sem percorrer os itens anteriores.

Listas pré-ordenadas e busca binária

```
alunosPorNome.Sort((a, b) => comparadorNome.Compare(a.Nome.Trim(),
b.Nome.Trim()));

alunosPorMedia.Sort((a, b) => b.Media.CompareTo(a.Media));


int meio = esquerda + (direita - esquerda) / 2;

int comparacao = comparadorNome.Compare(lista[meio].Nome.Trim(), nome);

if (comparacao == 0) return meio;

if (comparacao < 0) esquerda = meio + 1; else direita = meio - 1;
```

As listas são ordenadas uma única vez, em O(n log n). Alternar entre ordem por nome e por média passa a ser só trocar a lista exibida, em O(1). A busca binária descarta metade da lista a cada passo, em O(log n): com 1.000 alunos, são no máximo 10 comparações.

Dictionary por ID e por turma

```
Dictionary<int, Aluno> alunoPorId = new Dictionary<int, Aluno>();

Dictionary<string, List<Aluno>> alunosPorTurma = new Dictionary<string,
List<Aluno>>();

alunoPorId.TryGetValue(id, out var encontrado);
```

alunosPorTurma.TryGetValue(turma, out var listaDaTurma);

O Dictionary é uma tabela hash: encontrar um aluno pelo ID ou obter os alunos de uma turma custa O(1) em média. O agrupamento por turma é montado uma vez, em O(n).

Stack para a opção "Voltar"

historico.Push((listaAtual, tituloAtual, paginaAtual));

var anterior = historico.Pop();

Voltar à tela anterior segue a lógica LIFO (último a entrar, primeiro a sair). Push e Pop custam O(1).

Principais trechos do código

A seguir estão os trechos mais importantes do Program.cs, cada um com um resumo do que foi feito.

1. Paginação reutilizável (ObterPagina)

```
List<T> ObterPagina<T>(List<T> lista, int pagina, int tamanho)
{
    int inicio = (pagina - 1) * tamanho;
    if (inicio < 0 || inicio >= lista.Count) return new List<T>();
    int quantidade = Math.Min(tamanho, lista.Count - inicio);
    return lista.GetRange(inicio, quantidade);
}
```

O que foi feito: a paginação virou uma função genérica, usada para qualquer lista (todos os alunos, uma turma ou o ranking). Ela calcula a posição inicial da página e devolve só aqueles itens com GetRange, tratando página inválida e a última página incompleta.

2. Montagem dos dicionários (por ID e por turma)

```
Dictionary<int, Aluno> alunoPorId = new Dictionary<int, Aluno>();
foreach (var aluno in alunos)
{
    alunoPorId[aluno.Id] = aluno;
}

foreach (var aluno in alunosPorNome)
{
    string turma = aluno.Turma.Trim();
    if (!alunosPorTurma.TryGetValue(turma, out var listaDaTurma))
    {
        listaDaTurma = new List<Aluno>();
        alunosPorTurma[turma] = listaDaTurma;
    }
    listaDaTurma.Add(aluno);
}
```

O que foi feito: logo após ler o banco, os alunos são percorridos uma única vez para preencher dois dicionários. O primeiro liga cada ID ao seu aluno. O segundo separa os

alunos por turma: se a turma ainda não existe no dicionário, cria uma lista nova para ela. Como a lista percorrida já está em ordem alfabética, cada turma também fica ordenada.

3. Busca binária por nome

```
int BuscaBinariaPorNome(List<Aluno> lista, string nome)
{
    int esquerda = 0;
    int direita = lista.Count - 1;

    while (esquerda <= direita)
    {
        int meio = esquerda + (direita - esquerda) / 2;
        int comparacao = comparadorNome.Compare(
            lista[meio].Nome.Trim(), nome);

        if (comparacao == 0) return meio;
        if (comparacao < 0) esquerda = meio + 1;
        else direita = meio - 1;
    }
    return -1;
}
```

O que foi feito: a função compara o nome procurado com o aluno do meio da lista ordenada. Se o nome procurado vem depois, descarta a metade da esquerda; se vem antes, descarta a da direita. Repete até encontrar (devolve a posição) ou até não sobrar nada (devolve -1).

4. Opção de busca no menu: do resultado à página certa

```
int indice = BuscaBinariaPorNome(alunosPorNome, nome);

if (indice == -1)
{
    Console.WriteLine($"Aluno \"{nome}\" não encontrado.");
}
else
{
    SalvarEstado();
    listaAtual = alunosPorNome;
    paginaAtual = indice / pageSize + 1;
    alunoDestacado = alunosPorNome[indice];
}
```

O que foi feito: com a posição devolvida pela busca binária, o programa calcula direto em qual página o aluno está (posição dividida pelo tamanho da página) e já abre essa página com o aluno destacado. Antes disso, salva a tela atual na pilha para permitir voltar.

5. Filtro por turma e alternância de ordenação

```
// Opção 8 – filtrar por turma
if (alunosPorTurma.TryGetValue(turma, out var listaDaTurma))
{
    SalvarEstado();
    listaAtual = listaDaTurma;
    paginaAtual = 1;
}

// Opção 9 – alternar entre ordem por nome e por média
SalvarEstado();
if (listaAtual == alunosPorMedia)
    listaAtual = alunosPorNome;
else
    listaAtual = alunosPorMedia;
paginaAtual = 1;
```

O que foi feito: o filtro não percorre os alunos; só pede ao dicionário a lista daquela turma, que já está pronta, e passa a exibi-la. A troca de ordenação também não reordena nada: apenas alterna entre as duas listas que foram ordenadas no início.

6. Histórico de navegação com pilha (opção Voltar)

```
var historico = new Stack<(List<Aluno> Lista, string Titulo, int Pagina)>();

void SalvarEstado() => historico.Push((listaAtual, tituloAtual, paginaAtual));

// Opção 5 – Voltar
if (historico.Count > 0)
{
    var anterior = historico.Pop();
    listaAtual = anterior.Lista;
    tituloAtual = anterior.Titulo;
    paginaAtual = anterior.Pagina;
}
```

O que foi feito: toda vez que o usuário muda de tela (troca de página, busca, filtro), o estado atual é empilhado com Push. Ao escolher Voltar, o último estado é retirado com Pop e restaurado: a lista, o título e a página voltam a ser os de antes.